# Retrieval-Augmented Generation (RAG) → Perplexity AI




improve the LLM output

1. up to date
2. accuracy ↑
3. fact based ↑

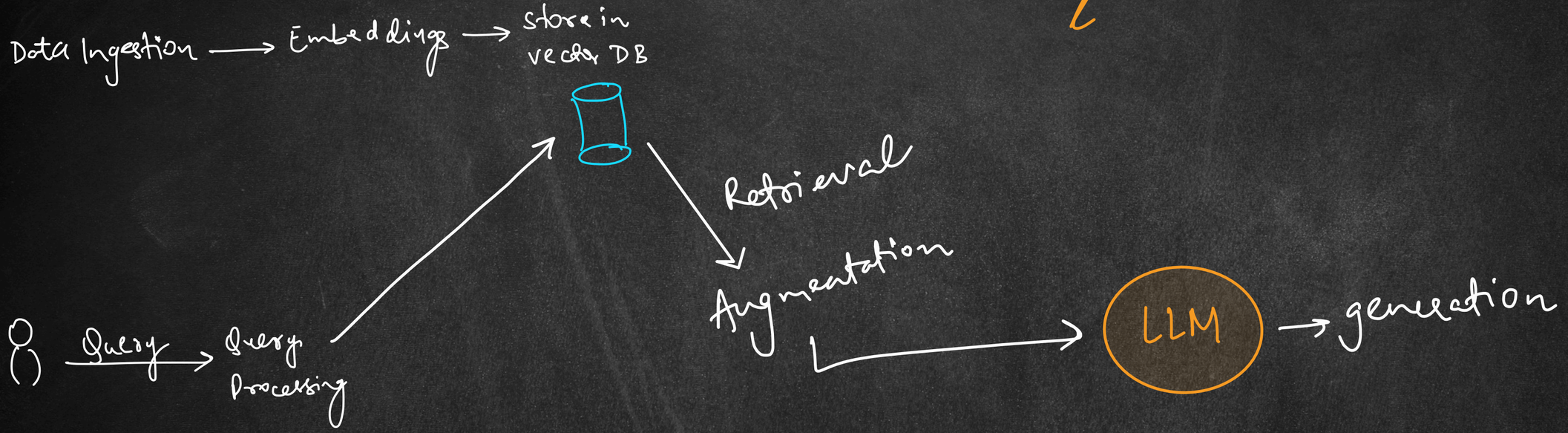
1. Hallucination — making things up
2. Not upto date

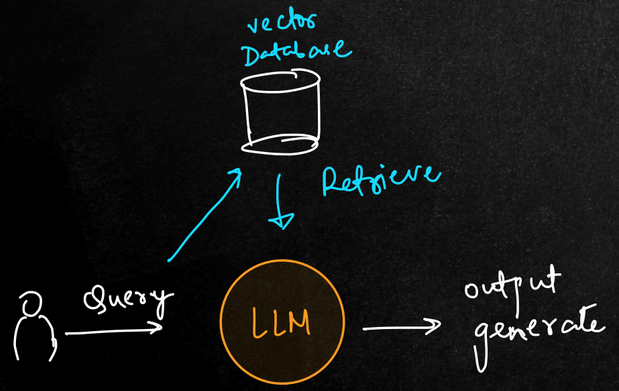
Information Retrieval

# RAG

## *Architecture*

RAG architecture




SQL - traditional DB based
name = "Harry Potter"

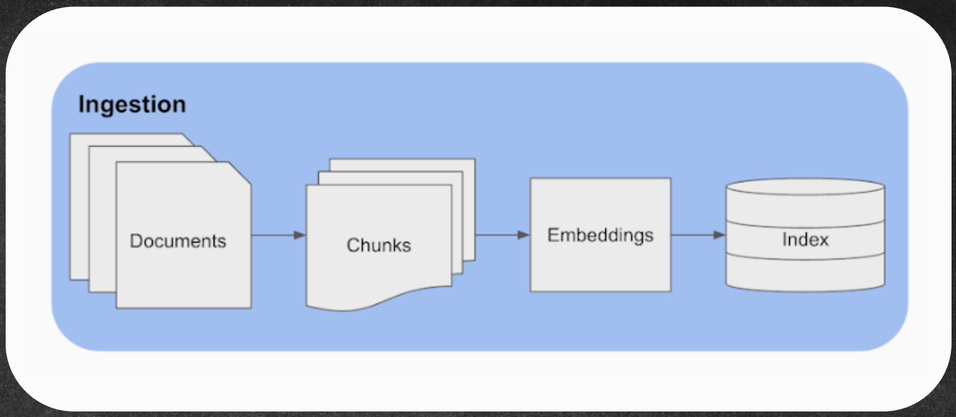
vector DB
→ wizards, children fantasy

# RAG

## Data Ingestion Pipeline

context size

LLM




text → nums

vector embeddings

① Data Ingestion → ② Embedding → ③ Vector DB → Knowledge Base

pdf
html
db
excel
⋮

⇓

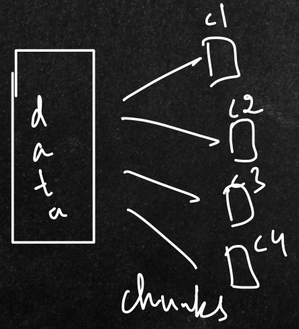
process/clean

⇓

chunks

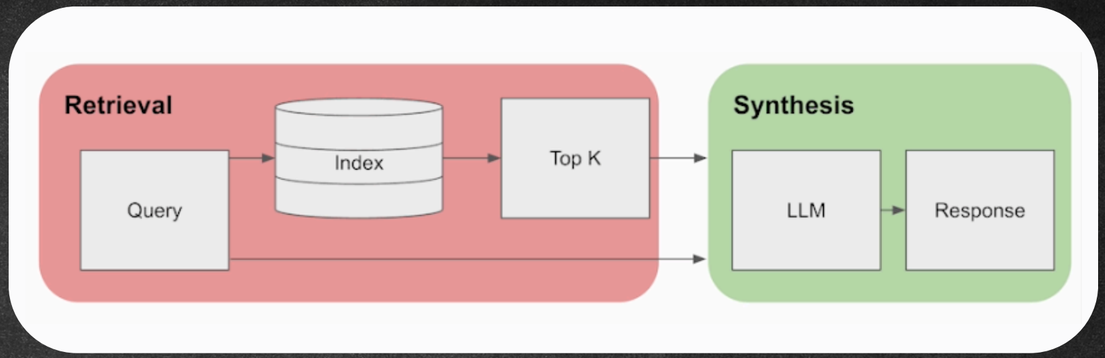
- OpenAI embedding models
- sentence transformer

Pinecone
FAISS
chromadb
⋮



# RAG

## Retrieval Pipeline




- Nearest neighbour search

- Cosine Similarity (vectors)

cosine dist ↓

cosine similarity = 1 − cosine dist

semantic / similarity search

① Retrieval

vector DB

④ Query Processing

Query Embeddings

Query

⑥ Augmentation

context

top K similar results

+ prompt

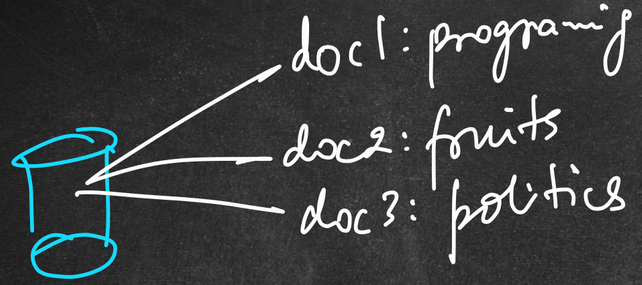
LLM

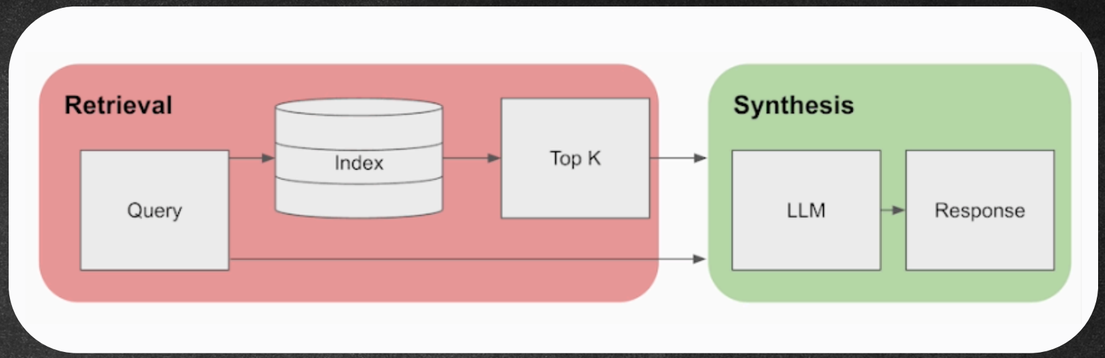
o/p generate

⑦ Generation

# RAG

## *Retrieval Pipeline*



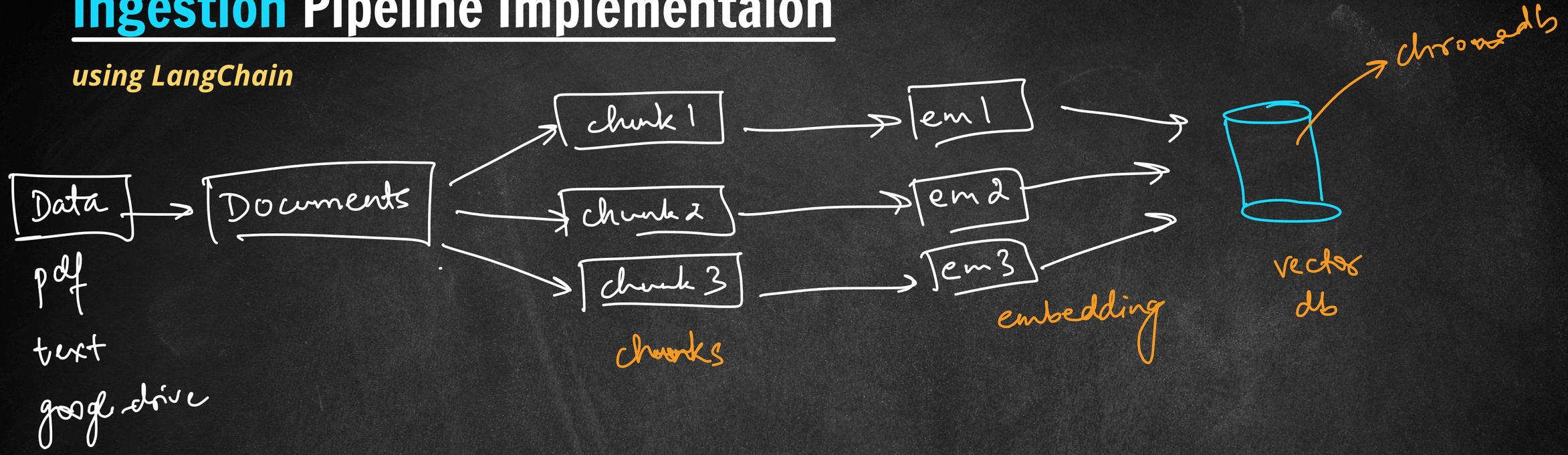




Query Processing → Retrival → Augmentation → Generation

→ Python

# Ingestion Pipeline Implementaion

*using LangChain*

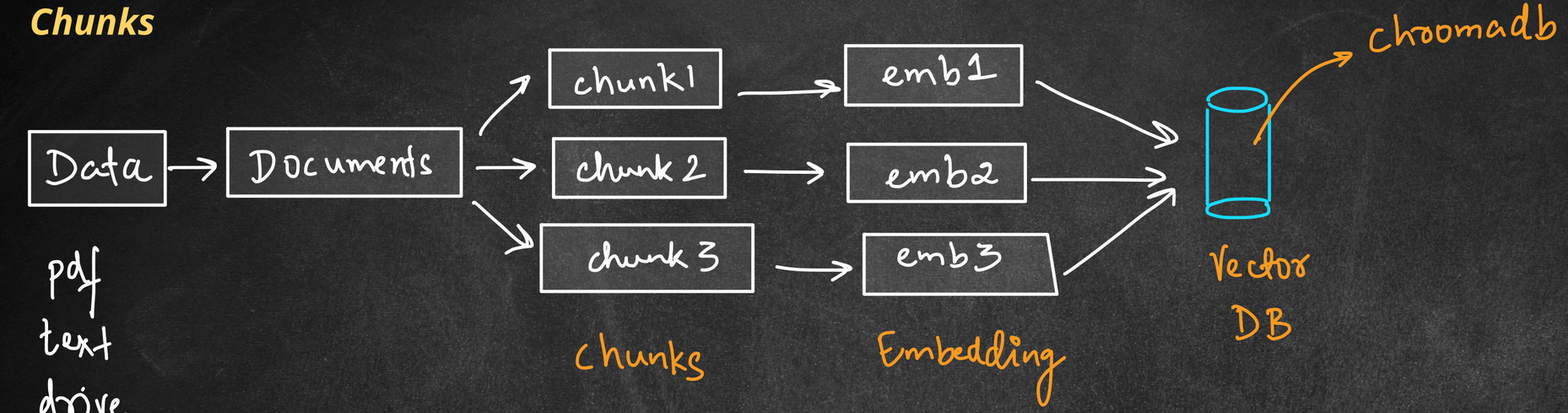



① Document → object
{
① text / page_content
② metadata
}

② Document Loaders

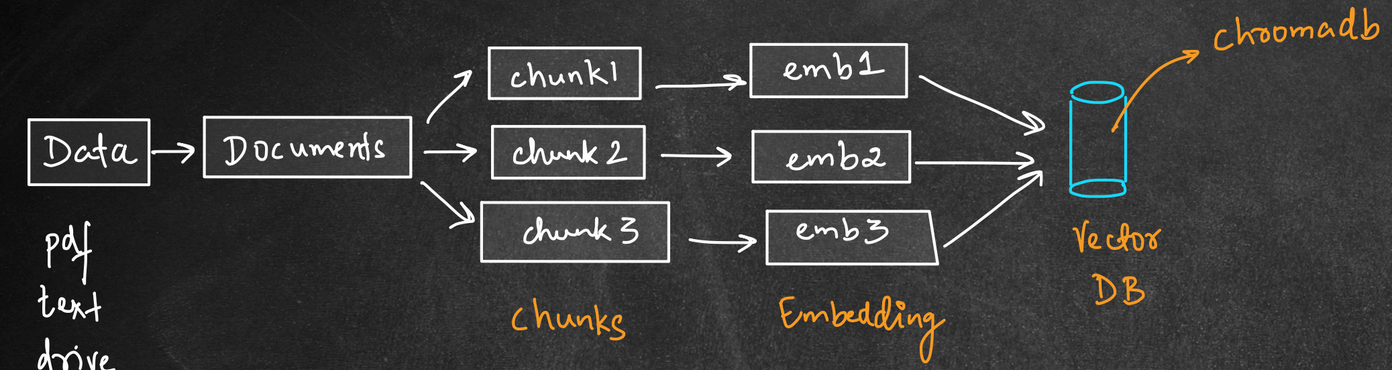
# Ingestion Pipeline Implementaion

***Chunks***



# Ingestion Pipeline Implementaion

***Embedding***




Embedding manager

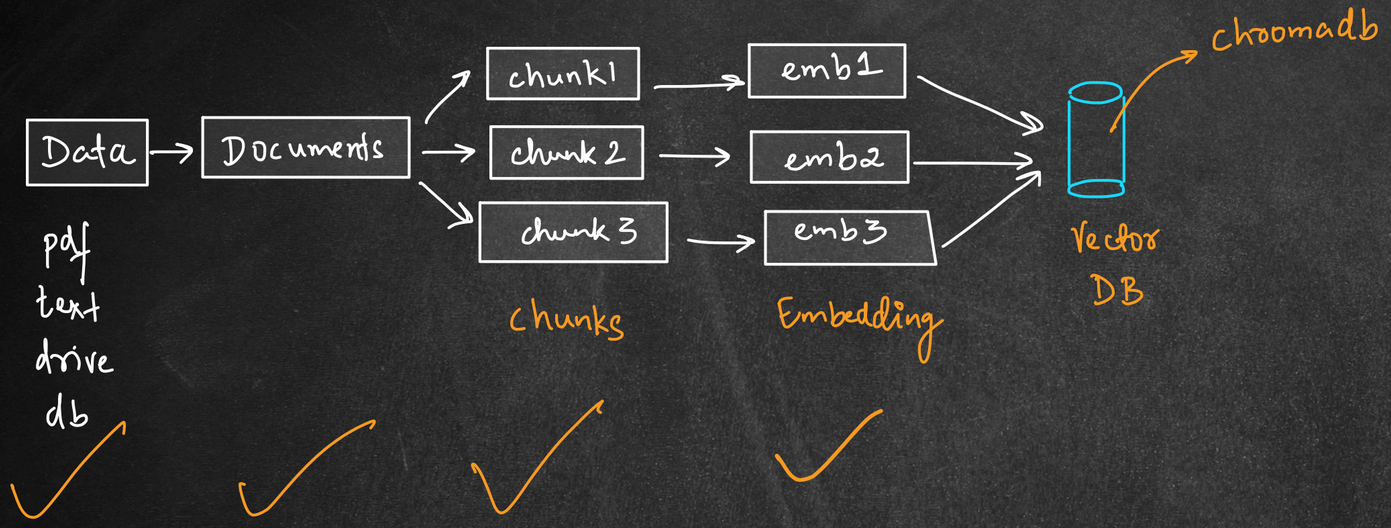
obj

code reusability ↑

Query Embedding

# Ingestion Pipeline Implementaion

## Vector Store

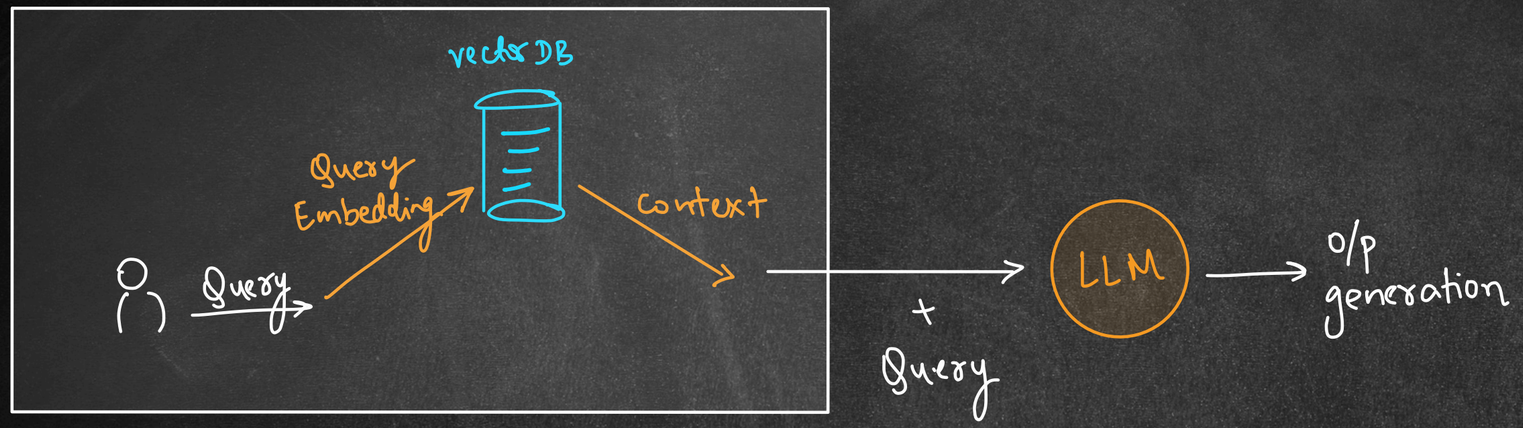



→ Vector Store Manager

1. initialize vector store — content
2. VS_ initialize → path, collection
3. store docs → VS

# Retrieval Pipeline Implementaion

***Retriever***




+ Query

LLM → o/p generation

RAG Retriever

return a list of results (context)